15 minutes past 7 = 7:15

20 minutes to 8 = 7:40

Quarter past 6 = 6:15

Quarter to 6 = 5:45.

half past 6 = 6:30.

noon or midday = 12:00 pm

midnight = 0:00 am.

&& > ||

↳ precedence of && is greater

*= < +

x *= (3+2)

↳ पहले ये होगा।

(*, /, %) > (+, -)

L→R L→R

```
public class main {
    public static int permute(int nums) {
        if(nums == 1)
            return 1;
        S.O.P(nums);
        int s = nums * permute(nums - 1);
        S.O.P(s + " called");
        return s;
    }
    public static void main(String[] args) {
        permute(5);
    }
}
```

o/p
5
4
3
2
2 called
6 called
24 called
120 called.

---

*ptr++ (→)

*++ptr (←)

++*ptr (←)

Toggling the character = (char)(ch ^ 32);

Bit manipulation

num & 1 == 0/1 even/odd

Swapping

a = 5, b = 7

a = a ^ b = 2

b = a ^ b = 5

a = a ^ b = 7

XOR

1) XOR with same number is always 0.

eg. 5 ^ 5 = 0

2) XOR with n with 0 is n

eg. 0 ^ n = n

1. Bit masking
2. find ith bit
3. set ith bit
4. Clear ith bit
5. find number of bits to change to convert a to b.

* LinkedHashMap may have one null key and multiple null values
* treemap tm1 = new TreeMap()

Java important

* Ch = 10 constant
if(ch != 'a' || ch != 'e' || ch != 'i' || ch != 'o' || ch != 'u')
↳ कभी भी इस तरह use न करें Answer नहीं आयेगा

always use ==

* int n = Integer.parseInt(br.readLine().trim()); → बचा दे (from safe of NumberFormat Exception)
char ch = 1;
* int c = ch;
तो यहाँ पर c Ascii रखता है।

* boolean Check[] = new boolean[]; → सारे को default initialize zero से कर देगा।

Boolean Check[] = new Boolean[];
↳ default initialize null.

* Object x = (int) a; ✗
Object x = (Integer) a; ✓

---

* int freq[] = new int[26];
Char str[] = { . . . }

freq[str[i] - 'a']++;
Char result = (Char) (i + 'a');

---

* for(Map.Entry<String, Integer> e : map.entrySet())
S.O.P ("key" + e.getKey() + "value: " + e.getValue())

* ContainsKey(key), containsValue(value), remove(')].

Time complexity : Time taken by an algorithm as a function of the length of Input.

```java
List<String> res = new ArrayList<>();
```

↳ convert res to String array

object को ही convert होगा Primitive में नहीं

```java
res.toArray(new String[res.size()]);
```

→ String[] ↳ ये हो जायेगा.

---

map मे index के basis पर नहीं निकाल सकते है।

```java
fun(int sum, int i) {
    sum += i;
    fun(sum+i, i)
}
```

→ इस दोनो मे बहोत बड़ा अन्तर है। Backtracking से समझो.

---

get value using key.

```java
map.get(key).
```

↳ यहाँ पर map.get(key)++

↳ करेंगे value को बढ़ाने के लिए तो error देगा. map.get(key)+1 → ✓

To find out key using value

```java
for(Map.Entry<Integer, Integer> entry : map.entrySet()){   // entrySet() → object है।
    if (entry.getValue() == 1)
        ---
        --
    }
}
```

✗ (entry.getKey())

Some important method of map

keySet(), value()

getKey() and getValue() iterator से होता है।

String x[] = a.split("\\+|i"); x[0]-real, x[1]=imagin.

Vowel को compare करने के लिए एक String में रखलो और फिर उसके contains() करके चेक करलो.

* Character.toUpperCase(c); / isLetter( ) / isDigit( ) (c → char)

* PriorityQueue के अन्दर element store किसी भी क्रम में हो सकता है परंतु निकालते वक्त priority wise निकलेगा (प्राथमिकता) by default min पहले क्योंकि Java में min heap में store होता है

add(), remove(), element().

---

factorial.

```
import java.math.*;
BigInteger num = new BigInteger("1");
for(int j = 1; j<=n; j++)
    num = num.multiply(BigInteger.valueOf(j));
```

~~कोई भी Array को सिधा Print करना हो तो तब ऐसेको~~
Arrays.toString(); Array को pass करना है.

Queue methods
- throws exception — add(), remove(), element()
- return false/null — offer() poll(), peek()

interface

```
Queue<Integer> q = new LinkedList<>();
Queue<Integer> q = new ArrayDeque<>();
q.offer(10);, q.poll(), q.peek(), q.size(), q.isEmpty()
```

---

to print all values in ascending order.

```
Map<Integer, Integer> m = new TreeMap<>();
m.values();
```

→ ये use करो क्योंकि ये Array and Caching use करता है।

/ इसमें भी साथ चलेगा (NaN के जगह पर infinity आएगा)

78.0 % 0.0 ⇒ NaN
78.0 % 0 ⇒ NaN
78 % 0 ⇒ Exception.
78 % 0.0 ⇒ NaN

NaN != NaN
true
infinity != infinity → false

```
Public class main {
    P S V m(String[] args){
        new Dog(1);
    3
    static class Dog {
        Public Dog(int i){
            i = 0;
        3
    3
3.
```

treeset में add करो

* Inheritance में method कुछ भी return कर सकता है यानि (base, Derived etc).
* यदि कोई data initialize नहीं करोगे और उसके साथ कोई भी operation करने पर यहाँ तक कि Print करने पर भी error देगा (Compilation error).

Greedy method says that a problem is solved in a stages

Job sequencing with Dead lines - Greedy Method.

$n = 5$

Jobs	J_1	J_2	J_3	J_4	J_5
Profits	20	15	10	5	1
dead lines	2	2	1	3	3

0 — J_2 — 1 — J_1 — 2 — 3

Job Considered	slot assign	solution	Profit
-	-	-	0
J_1	[1,2]	J_1	20
J_2	[0,1] [1,2]	J_1, J_2	20 + 15
J_3 X	[0,1] [1,2]	J_1, J_2	20 + 15
J_4	[0,1] [1,2] [2,3]	J_1, J_2, J_4	20 + 15 + 5
J_5	"	"	40

$n = 7$

Jobs	J_1	J_2	J_3	J_4	J_5	J_6	J_7
profits	35	30	25	20	15	12	5
dead lines	3	4	4	2	3	1	2

0 — J_4 — 1 — J_3 — 2 — J_1 — 3 — J_2 — 4

20 25 35 30 = 110

Branch and bound is used to solve minimization problem.

Job sequencing with dead line with Branch and Bound.

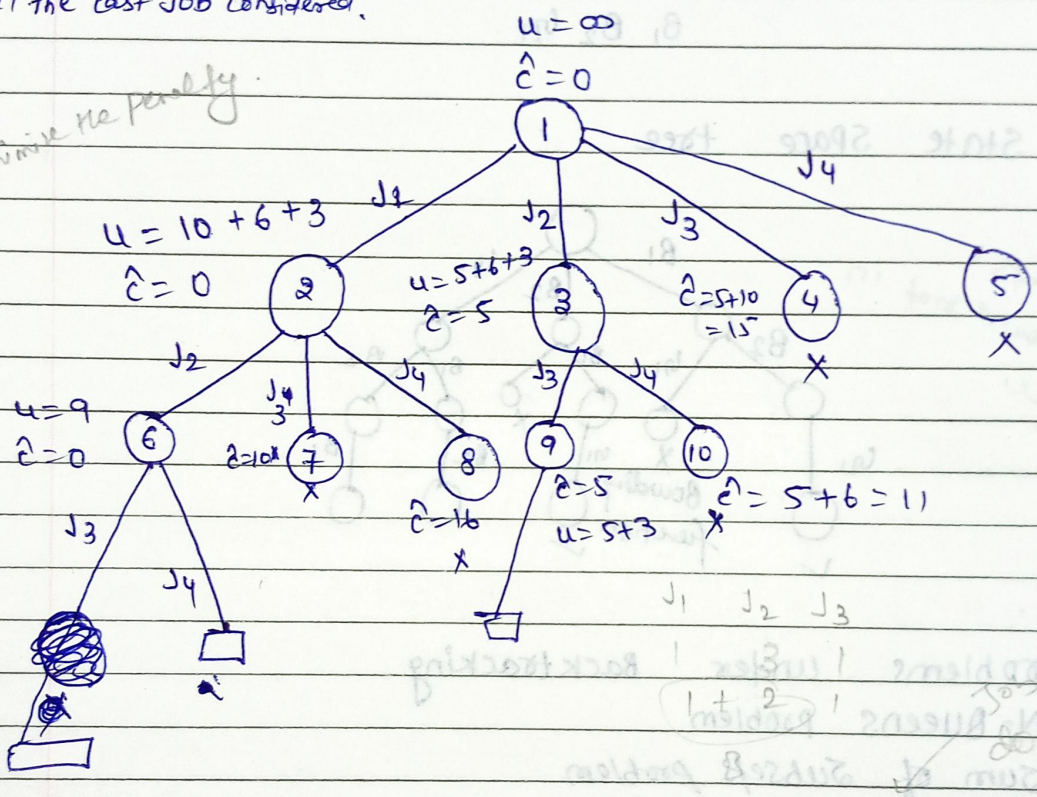
	1	2	3	4
Penalty	5	10	6	3
deadline	1	3	2	1
Time	1	2	1	1

u = Sum of all Penalties except that included in Sol.

$\hat{c}$ = Sum of penalties till the last Job considered.

Upper = ∞, 19, 14, 9, 8

minimize the penalty.



J_1 J_2 J_3

$\hat{c} = 5$ J_2, J_3

$u = 8$ 10 6 = 16

X J_1, J_4

5, 3 = 8

BFS
⇑
Branch and Bound (also use the strategy of Brute Force)

it is not for optimization problem. it's used when we want all possible solution.

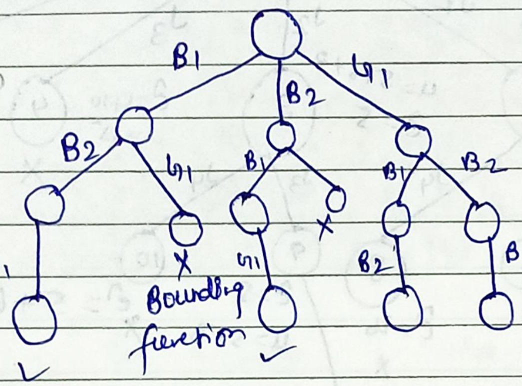
BackTracking · (it uses) Brute Force Approach. (DFS)

* Brute force approach says that for any given problem we try out all possible solution and pick up desired solution.

B_1 B_2 G_1

State Space tree

constraints: Girls should not in middle



Problems under Backtracking
1. N-Queens problem
2. Sum of subsets problem
3. Graph Coloring problem

Greedy method

it is used to solve optimization problem (min or max)

* Feasible solution:- A solution which satisfy some constraints.

2. Dynamic Programming
3. Branch and Bound

Recurrences :- when an algorithm contains a recursive call to itself its running time can be describe by recurrence equation.

eg.

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T(n/2) + n & \text{if } n > 1 \end{cases}$$

1.
```
fib(n)
    if(n<= 1)
        return 1;
    else
        return fib(n-1) + fib(n-2)
```

Solution

Solving Recurrences :

There are three method to solve Recurrence

1. Substitution method
2. Recursion Tree method
3. Master method

1. Substitution method (Back Substitution method)

$$T(n) = \begin{cases} 1 & \text{if } n = 0 \\ T(n-1) + n & \text{if } n > 0 \end{cases}$$

Ans. $O(n^2)$.

2. Recursion tree Method.

eg.

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T\left(\frac{n}{2}\right) + n^2 & \text{if } n > 1 \end{cases}$$

• The different cases of master's theorem to solve recurrences are:
not exhaustive

Master Method

The problem is divided into number of subproblems each of size $(\frac{n}{b})$ and need time $f(n)$ to combine the solution. then the running time $T(n)$ can be

$$T(n) = aT(\frac{n}{b}) + f(n)$$

where

$a \geq 1$, $b > 1$ $f(n)$ is asymptotically positive function

$\frac{n}{b}$ means $\lfloor \frac{n}{b} \rfloor$ or $\lceil \frac{n}{b} \rceil$

$f(n) = \theta(n^k \log^p n)$

$T(n)$ can be bounded asymptotically as follows

1. $f(n) < n^{\log_b^a}$ or $f(n) = O(n^{\log_b^a - P})$ for some constant $P > 0$ then $T(n) = \theta(n^{\log_b^a})$

2. $f(n) = n^{\log_b^a}$ or $f(n) = \theta(n^{\log_b^a})$ then $T(n) = (n^{\log_b^a} \cdot \log n)$

3. $f(n) > n^{\log_b^a}$ or $f(n) = \Omega(n^{\log_b^a + P})$ for some constant $P > 0$ and if $a \cdot f(\frac{n}{b}) \leq c \cdot f(n)$ some constant $c < 1$ and all sufficiently large n, then

$$T(n) = \theta(f(n))$$

if $P > -1$ $\theta(n^k \log^{P+1} n)$

if $P = -1$ $\theta(n^k \log \log n)$

if $P < -1$ $\theta(n^k)$

if $P \geq 0$ $\theta(n^k \log^p n)$

if $P < 0$ $\theta(n^k)$.

case 1

eg. $T(n) = 2T(n/2) + 1$

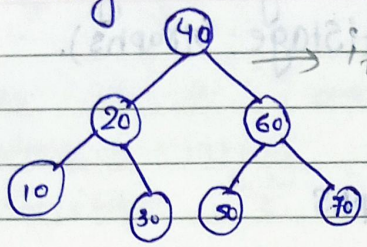
$a = 2, b = 2$ $f(n) = \theta(1) = \theta(n^0 \log^0 n)$

$k = 0, P = 0$

$\theta(n)$ Ans

Optimal Binary Search tree

cost of BST is minimum



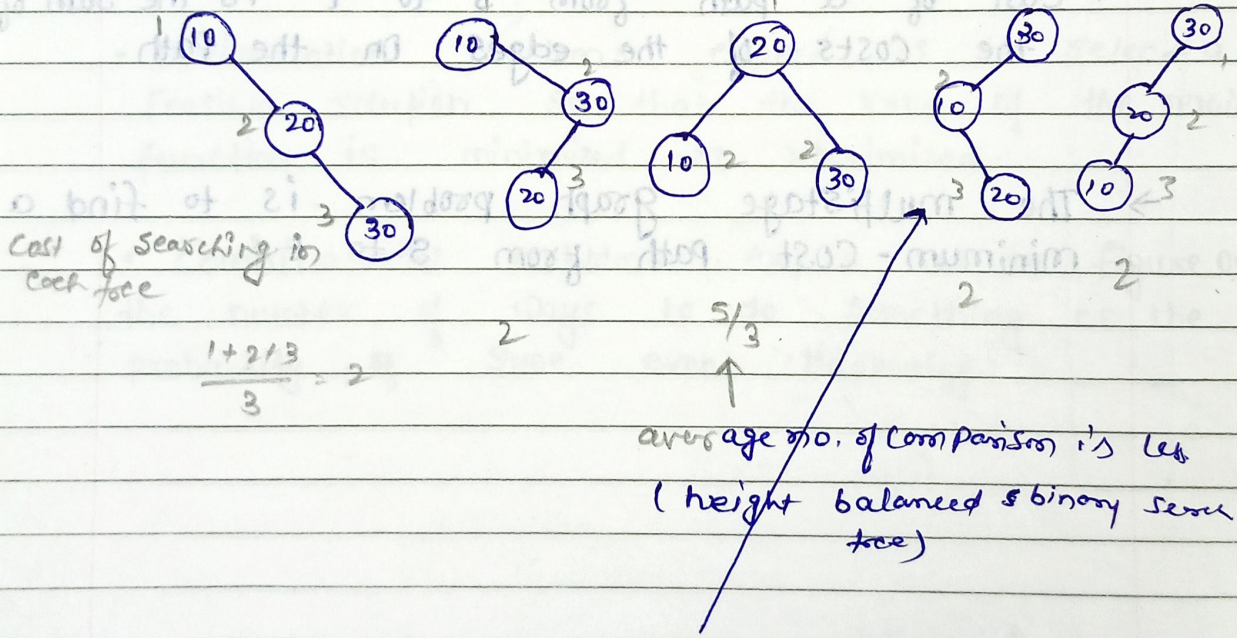
→ it is helpful in searching

Total no. of Binary Search tree are possible for n node element

$$\frac{2n C_n}{n+1}$$

$n = 3$

$$\frac{2*3 C_3}{3+1} = 5$$



Cost of searching in each tree

$\frac{1+2+3}{3} = 2$

2

5/3

2

2

average no. of comparison is less (height balanced & binary search tree)

if frequency are given then cost is less for this tree.

	1	2	3	4
Keys	10	20	30	40
freq	4	2	6	3

i \ j	0	1	2	3	4
0	0	4	8	20	26
1		0	2	10	16
2			0	6	12
3				0	3
4					0

root

① $l = j - i = 0$

$0-0=0$, $1-1=0$, $2-2=0$, $3-3=0$ $4-4=0$

② $l = j - i = 1$

$1-0=(0,1)$ $2-1=1\ (1,2)$ $3-2=1\ (2,3)$ $4-3=1\ (3,4)$.

$C(0,1) \rightarrow 4$ ← cost

$C[1,2] \rightarrow 2$ ← ignores, take this

$C[2,3] \rightarrow 6$

$C[3,4] \rightarrow 3$

10	20	30	40
4	2	6	3

$l = j - i = 2$

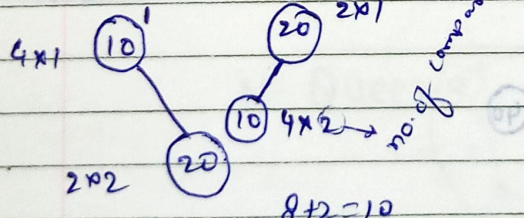
$2-0=(0,2)$ $3-1=(1,3)$ $4-2=(2,4)$

$w(0,4) = \sum_{i=1}^{4} f(i)$

1st and 2nd key.

$C[0,2]$

10	20
4	2

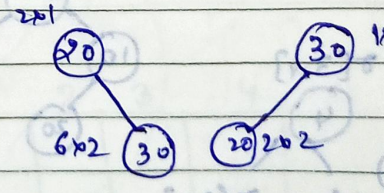


$4+4=8$ ← minimum

$8+2=10$

$C[1,3]$

20	30
2	6

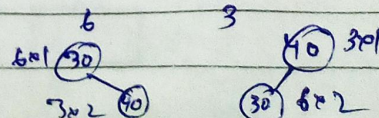


$2+12=14$

$4+6=10$ ← minimum

$C[2,4]$

3	4
30	40
6	3



$C[0,0] + C[1,2] + w[0,2]$

$0 + 2 + 4+2 = 8$

$C[0,1] + C[2,2] + w[0,2] = 4+0+6 = 10$

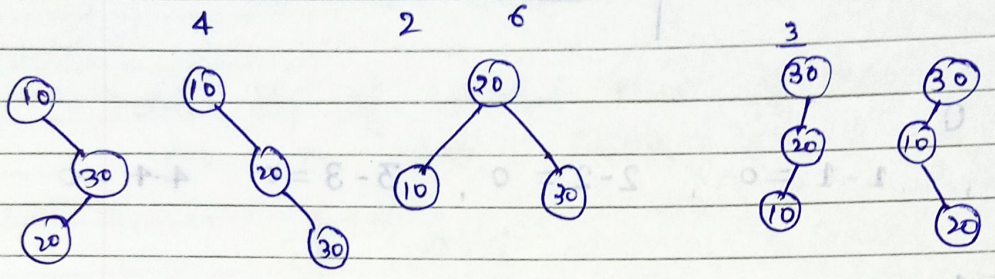
$l = j - i = 3$

$3 - 0 = (0,3)$ $\quad 4 - 1 = (1,4)$

$c[0,3]$

1	2	3
10	20	30
4	2	6

$w[0,3] = 12$



$c[0,3] \min \{ c(0,0) + c[1,3] + 12, \; c[0,1] + c[2,3] + 12, \; c[0,2] + c[3,3] + 12 \}$

$22 \qquad 22 \qquad 20$

$c[1,4] \qquad w[1,4] = 11$

20	30	40
2	6	3

$c[1,4] = \min \{ c[1,1] + c[3,4] + 12, \; c[1,2] + c[3,4] + 11, \; c[1,3] + c[4,4] + 11 \}$ (1 is a root)

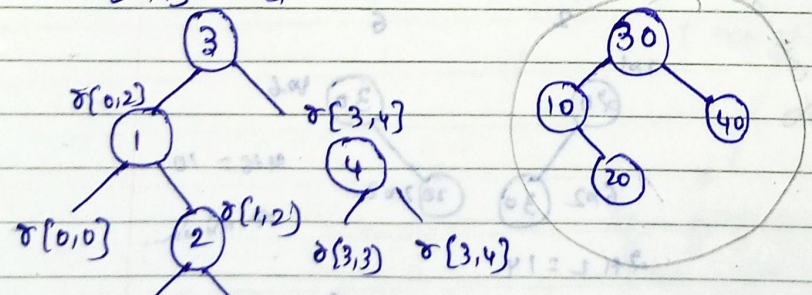
$l = j - i = 4 \qquad 4 - 0 = (0,4) \qquad w[0,4] = 15$

$c[0,4]$

1	2	3	4
10	20	30	40
4	2	6	3

$c[0,4] \min \{ c(0,0) + c[1,4], \; c[0,1] + c[2,4], \; c[0,2] + c[3,4], \; c[0,3] + c[4,4] \} + 15$ → $8+3+15$

$r[0,4] = 3$



$$c[i,j] = \min_{i<k\le j} \{ c[i,k-1] + c[k,j] \} + w(i,j)$$

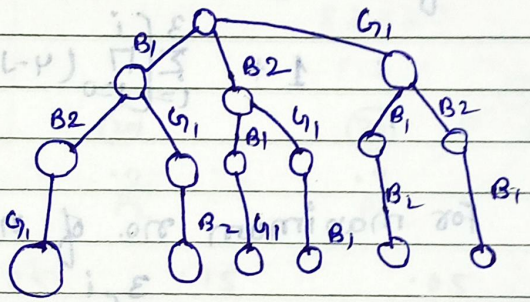
Backtracking → it follows DFS
→ it is not for optimization problem
→ it follows the brute force approach
→ It gives every possible solution

A general algorithmic technique that considered searching every possible combination in order to solve a computational problem.

it is used when you have multiple solution and you want all those solution

B_1 B_2 G_1

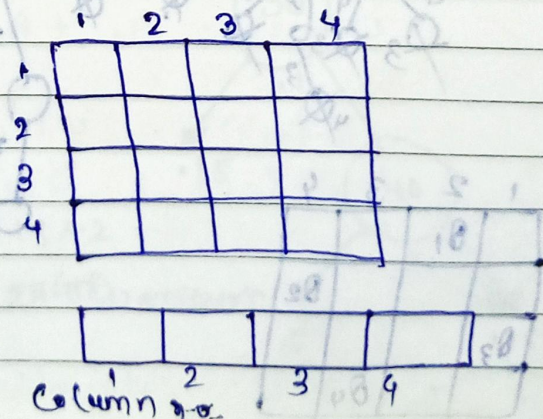
State Space tree



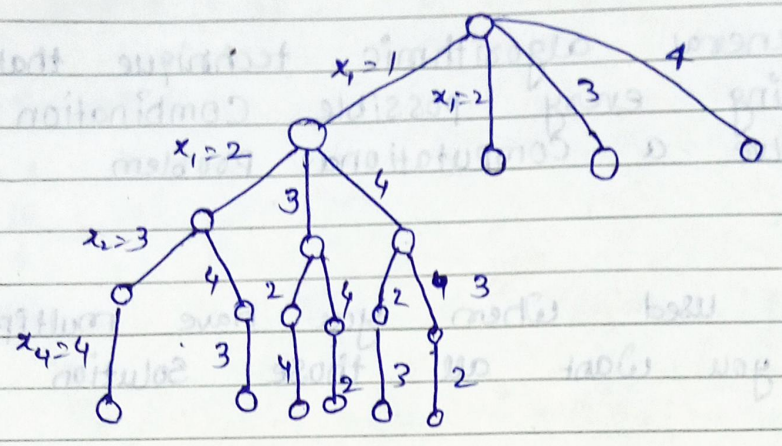
bounding function
↓
for apply a constraints

Branch and Bound → it follows BFS

N- Queens



State Space tree



Bounding function
यानि Leaf नहीं

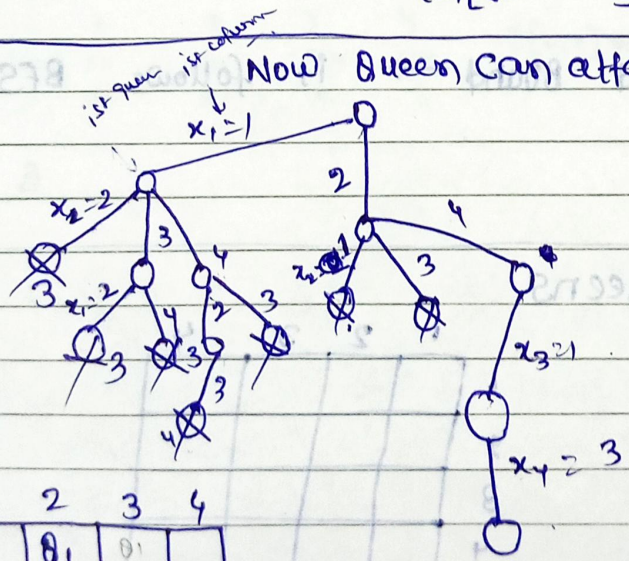
No. of nodes $1 + 4 + 4\times3 + 4\times3\times2 + 4\times3\times2\times1$

$$1 + \sum_{i=1}^{3}\left[\prod_{j=0}^{i}(4-j)\right] = 65$$

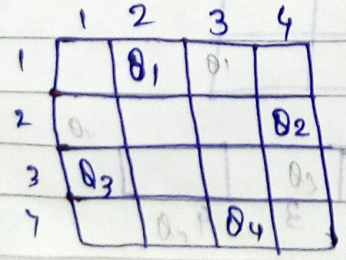
For maximum no. of nodes

$$1 + \sum_{i=1}^{3}\left[\prod_{j=0}^{i}(N-j)\right]$$

Now Queen can attack diagonally



Solution
2 4 1 3
3 1 4 2



Mirror image

x

2	4	1	3
1	2	3	4

cost is minimum — level always start from 1 in B.T.

Optimal Binary Search tree

* The amount of Comparison that we are doing and that amount of Comparison is called cost of searching in B.T.

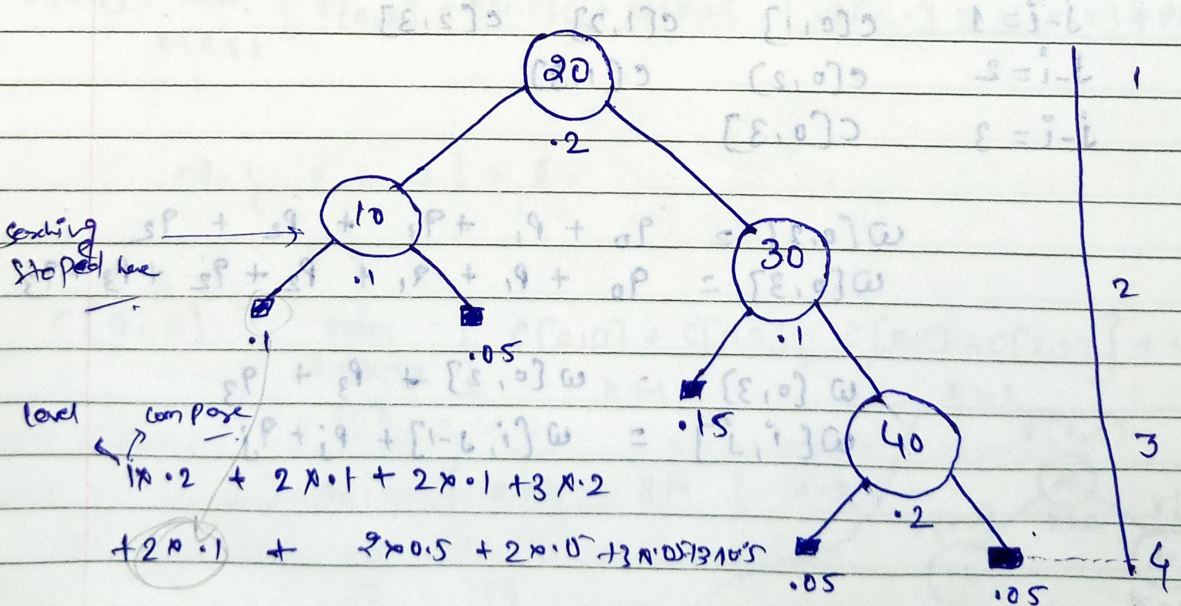
* null/dummy nodes represent the unsucessful search

* no. of level is equal to the maximum search

* no. of square node/null nodes/dummy nodes = no. of node+1

Keys		10		20		30		60	
p_i		.1 (p_1)		.2 (p_2)		.3 (p_3)		.2 (p_4)	
q_i	.1 (q_0)		.05 (q_1)		.15 (q_2)		.05 (q_3)		.05 (q_4)

$-\infty$ to 9   61 – ∞



level compose

$1\times.2 + 2\times.1 + 2\times.1 + 3\times.2$

$+ 2\times.1 + 2\times0.5 + 2\times.05 + 3\times.05 + 3\times.05$

= 2.1 → cost of searching in this BT (considering successful and unsuccessful search).

cost is minimum you have to found out

$$Cost[0,n] = \sum_{1 \le i \le n} p_i \times level(a_i) + \sum_{0 \le i \le n} q_i * (level(E_i) - 1)$$

Optimal Binary Search Tree using Dynamic Programming

try out all indirectly to find best possible minimum cost BST

$$c[i,j] = \min_{i < k \le j} \{ c[i,k-1] + c[k,j] \} + \omega[i,j]$$

$$c[0,3] = \min_{0 < k \le 3} \{ c[0,0] + c[1,3],\ c[0,1] + c[2,3],\ c[0,2] + c[3,3] \} + \omega[0,3]$$

$k = 1, 2, 3$

$$c[1,3] = \min_{1 < k \le 3} \{ c[1,1] + c[2,3],\ c[1,2] + c[3,3] \} + \omega[1,3]$$

$k = 2, 3$

	i		j
$j-i=0$	$c[0,0]$	$c[1,1]$	$c[2,2]$ $c[3,3]$
$j-i=1$	$c[0,1]$	$c[1,2]$	$c[2,3]$
$j-i=2$	$c[0,2]$	$c[1,3]$	
$j-i=3$	$c[0,3]$		

$$\omega[0,2] = q_0 + p_1 + q_1 + p_2 + q_2$$
$$\omega[0,3] = q_0 + p_1 + q_1 + p_2 + q_2 + p_3 + q_3$$

$$\omega[0,3] = \omega[0,2] + p_3 + q_3$$
$$\omega[i,j] = \omega[i,j-1] + p_j + q_j$$

Dynamic Programming follows tabular approach

	0	1	2	3	4
	10	20	30	40	
p_i =		3	3	1	1
q_i =	2	3	1	1	1

$w[0,0] = q_0$ + —
$w[1,1] = q_1$ + —
$w[2,2] = q_2$ + —
$w[3,3] = q_3$ + —
$w[4,4] = q_4$ + —

Divide by 16 to get the probability

	0	1	2	3	4
$j-i=0$	$w_{00}=2$ $c_{00}=0$ $r_{00}=0$	$w_{11}=3$ $c_{11}=0$ $r_{11}=0$	$w_{22}=1$ $c_{22}=0$ $r_{22}=0$	$w_{33}=1$ $c_{33}=0$ $r_{33}=0$	$w_{44}=1$ $c_{44}=0$ $r_{44}=0$
$j-i=1$	$w_{01}=8$ $c_{01}=8$ $r_{01}=1$	$w_{12}=7$ $c_{12}=7$ $r_{12}=2$	$w_{23}=3$ $c_{23}=3$ $r_{23}=3$	$w_{34}=3$ $c_{34}=3$ $r_{34}=4$	→ min of k value
$j-i=2$	$w_{02}=12$ $c_{02}=19$ $r_{02}=1$	$w_{13}=9$ $c_{13}=12$ $r_{13}=2$	$w_{24}=5$ $c_{24}=8$ $r_{24}=3$		
$j-i=3$	$w_{03}=14$ $c_{03}=25$ $r_{01}=2$	$w_{14}=11$ $c_{14}=19$ $r_{14}=2$			
$j-i=4$	$w_{04}=16$ $c_{04}=32$ $r_{04}=2$				

$\frac{32}{16} = 2$ → the min cost is 2

$$c[i,j] = \min_{i<k\le j} \{ c[i,k-1] + c[k,j] \} + w[i,j]$$

$$c[0,1] = \min_{0<k\le 1} \{ c[0,0] + c[1,1] \} + w[0,1]$$

$k = 1$

$\min\{ 0 + 0 \} + 8$

$= 8$

$w[0,0] = q_0$
$w[1,1] = q_1$
$w[0,1] = w[0,0] + p_i + q_j$

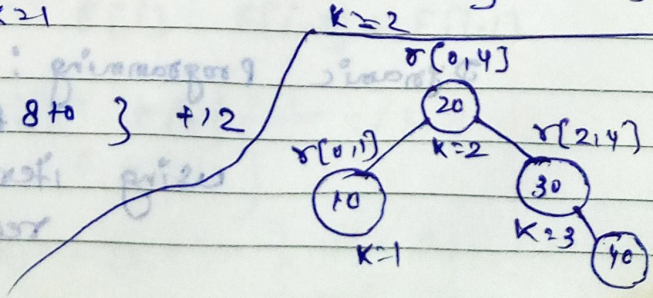
$i \quad j$

$$c[0,2] = \min_{0<k\le 2} \{ c[0,0] + c[1,2],\ c[0,1] + c[2,2] \} + w[0,2]$$

$k = 1, 2$

$\min\{ 0+7,\ 8+0 \} + 12$

$= 19$



Dynamic programming is used for solving of optimization problem - (greatest or minimum solution).

→ in greedy method we follow the predefined precession to get the solution of optimization problem.

Greedy method may be give the optimization problem or may not be but Dynamic programming always give the optimized problem

Dynamic programming: It divide the problem into series of overlapping subproblems.

Two features.

① optimal substructure

② overlapping subproblems.

in divide and conquer that not exist

Fibonacci series

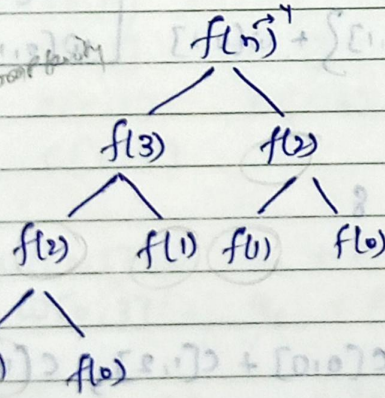
n =	0	1	2	3	4	5		
f(n) =	0	1	1	2	3	5	8	13

$$f(n) = f(n-1) + f(n-2)$$

$T(n) = 2T(n-1) + 1$

$O(2^n)$

to reduce the time complexity we use DP



Program may not be recursive but formula will be recursive

Dynamic Programming: it is solved by recursive formula. Programming: mostly it is solved using iteration or we can also use recursive.

D.P follow principle of optimality.

$$fib(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ fib(n-2) + fib(n-1) & \text{if } n>1 \end{cases}$$

initially all fill with -1

F =

0	1	1	2	3	5
0	1	2	3	4	5

$\theta(n)$ memoization

fib(n) = n+1 calls

T(n) = n

it reduces the time complexity from (2^n to n).

memoization follows top-down approach.

using iterative

$$fib(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ fib(n-2) + fib(n-1) & \text{if } n>1 \end{cases}$$

```
int fib(int n)
{
    if (n <= 1)
        return n;

    f[0] = 0;  f[1] = 1;
    for( int i=2; i<=n; i++)
        F[i] = F[i-2] + F[i-1]

    return F[n];
}
```

it follows bottom up approach

from zero we start

iteratively we find n

f

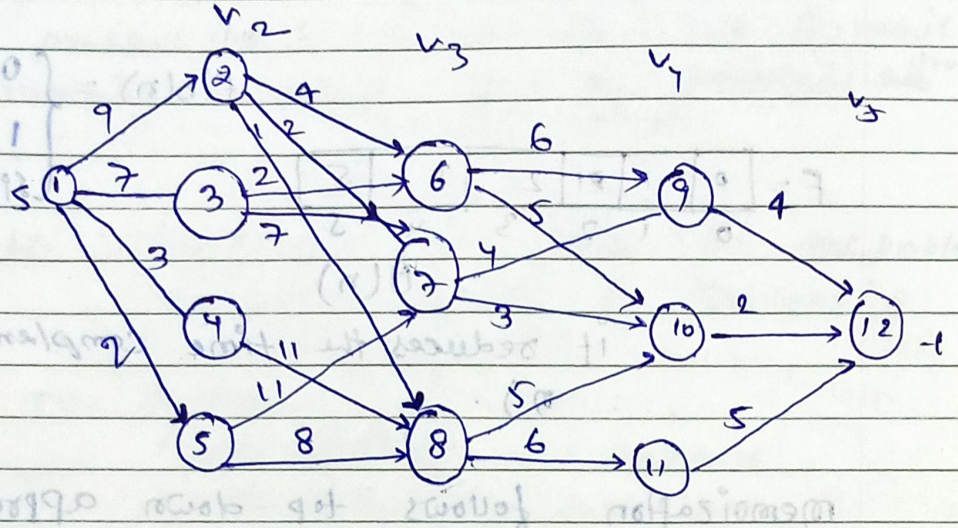
0	1	1	2	3	5
0	1	2	3	4	5

recursion top to down

iteration bottom to top

→ forward method

Multistage graph.



v	1	2	3	4	5	6	7	8	9	10	11	12
cost	16	7	9	18	15	7	5	7	4	2	5	0
d	2/3	7	6	8	8	10	10	10	12	12	12	12

stage no. vertex no.

Cost(5, 12) = 0

Cost(4, 9) = 4

Cost(4, 10) = 2

Cost(4, 11) = 5

Cost(3, 6) = min{ cost(6,9) + cost(4,9), cost(6,10) + cost(4,10) }

{6+4, 5+2} = 7

Cost(3, 7) = min{ cost(7,9) + cost(4,9), cost(7,10) + cost(4,10) }

4 + 4 , 3 + 2 = 5

Cost(3, 8) = min{ cost(8,10) + cost(4,10), cost(8,11) + cost(4,11) }

5 + 2 , 6 + 5 = 7

c(2,2) = min{ c(2,6) + c(6,6), cost(2,7) + c(7,7), c(2,8) + c(8,8) }

4 + 7 , 2 + 5 , 1 + 7 = 7

c(2,4) = { c(4,8) + c(8,8) } 11 + 7 = 18

Formula: stage no. → i, vertex no. → j

$$Cost(i, j) = \min_{l \in V_{i+1},\ \langle j, l \rangle \in E} \{ Cost(j, l) + Cost(i+1, l) \}$$

stage no. → 1, vertex no. → 1

$$d(1, 1) = 2$$

$$d(2, 2) = 7$$
$$d(3, 7) = 10$$
$$d(4, 10) = 12$$

1 to 2 to 7 to 10 to 12 → minimum path

1 to 3 to 6 to 10 to 12

Minimum Path using DP.

Branch and Bound → BFS. Used only for solving minimization problem

Job sequencing

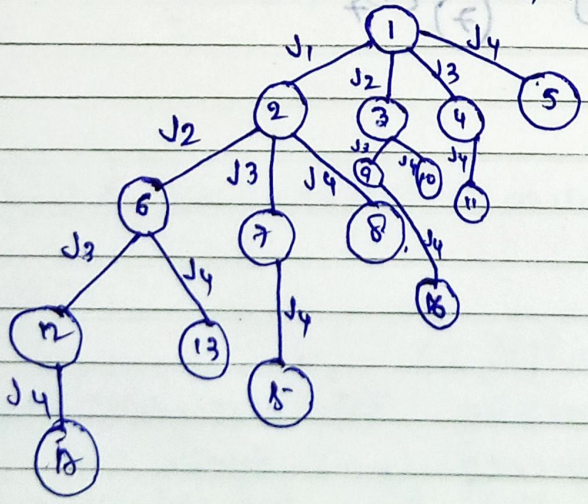
Question

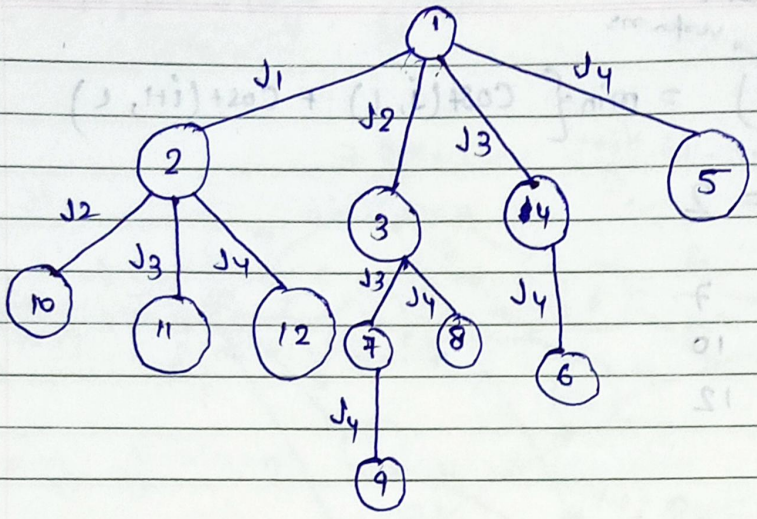
Job = { J1, J2, J3, J4 }
P = { 10, 5, 8, 3 }
d = { 1, 2, 1, 2 }

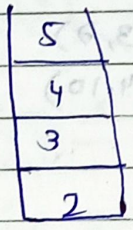
Soln

$S_1 = \{J_1, J_4\}$ ✓

$S_2 = \{1, 0, 0, 1\}$





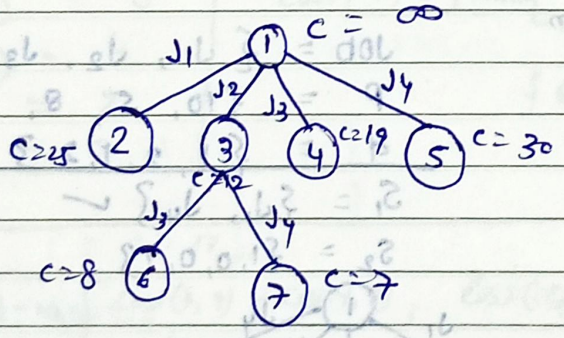


Queue → FIFO

Stack → LIFO

Least Cost - LC - BB

Least - Cost - BB



N-Queens Problem

- Generalisation of the 4-queens problem to n-queens by considering an nxn chessboard
  - Find all ways to place n non-attacking queens.

- APPROACH
  - From the 4-queens problem we observe that if $(S_1, S_2, \cdots S_n)$ is a solution vector where S_i is the column of the ith row where the ith queen is placed then

  - How do we test whether two queens are on the same diagonal

    - If two queens are placed at position (i,j) and (k, l), then they are on the same diagonal if

$$i - j = k - l \quad \text{or} \quad i + j = k + l$$

  - Therefore, two queens lie on the same diagonal iff $|j-l| = |i-k|$

N-Queens Problem : Generalized Backtracking Algorithm

```
Algorithm NQueens(k, n)
// using backtracking, this procedure prints all
// possible placements of n queens on an nxn
// chessboard so that they are non-attacking
```

```
{
    for := 1 to n do
    {
        if Place(k, i) then
        {
            S[k] := i ;
            if (k == n) then write(S[1:n]);
            else
                NQueens(k+1, n);
        }
    }
}
```

```
Algorithm Place(k, i)
// Return true if a queen can be place in the kth
// row and ith column. Otherwise it returns false.
// S[] is a global array whose first (k-1) values
// have been set Abs(r) returns the absolute value
// of r
  {
                    same column          same diagonal
    for j := 1 to k-1 do
      if((S[j] == i) or (Abs(S[j] - i) == Abs(j-k)))
    then
        return false

    return true ;

}
```